IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts Science Commerce and Management

Project – Low Level Design

on

Natural Language Interface for Manufacturing Infrastructure

Student Name: Anaina Vadakkanchery

Industry Mentor: Mr.Shashank **Guided by: Mr.Praveen**

Table of Contents

1. Introduction……3
   1.1. Scope of the document……3
   1.2. Intended Audience……...3
   1.3. System overview……..4

2. System Design……4
   2.1. Application Design……..4
   2.2. Process Flow……6
   2.3. Information Flow……7
   2.4. Components Design……8
   2.5. Key Design Considerations……..9
   2.6. API Catalogue……..9

3. Data Design……9
   3.1. Data Model……..9
   3.2. Data Access Mechanism……11
   3.3. Data Retention Policies……..11
   3.4. Data Migration……...11

4. Interfaces……...11

5. State and Session Management……..12

6. Caching……..12

7. Non-Functional Requirements……...12
   7.1. Security Aspects……..12
   7.2. Performance Aspects……...13

8. References……13

1. Introduction

This Low Level Design document provides a detailed technical description of how each component of InfraChat is implemented internally. While the HLD describes the overall architecture, this document covers specific functions, logic flows, data structures and interface definitions that make up the actual implementation.

1.1 Scope of the Document

This document covers the complete low level design of all components within the InfraChat system. It includes detailed descriptions of all Python backend functions in app.py, the step by step process flow for chatbot query handling, the string matching algorithm with its two-layer fuzzy matching approach, the risk scoring algorithm with its threshold conditions, the complete dataset structure and entity relationships, the data access mechanism using Pandas, data retention policies, data migration considerations, all system interfaces, state and session management, caching strategy and non functional requirements covering security and performance.

This document does not cover the high level architecture which is described in the HLD document. It does not cover production deployment or future database integration.

1.2 Intended Audience

- Internal project guide and industry mentor
- Project evaluation panel members
- Developers who want to understand the overall system architecture
- Students studying natural language processing and web development

1.3 System Overview

InfraChat is a web-based natural language interface for manufacturing infrastructure management that allows users to query machine data using simple English without technical knowledge.

The system has two interfaces: a homepage with a floating chatbot and a dashboard showing charts, KPIs, and a data table.

The core logic is the get_response() function, which uses RapidFuzz for fuzzy matching to handle user queries and spelling mistakes. It also supports machine-specific queries using Product IDs to fetch sensor data and calculate risk scores.

The system uses a Kaggle machine maintenance dataset with 10,000 records, including features like machine type, speed, torque, tool wear, and failure status. The dataset is loaded into a Pandas DataFrame at startup and stored in memory for fast access.

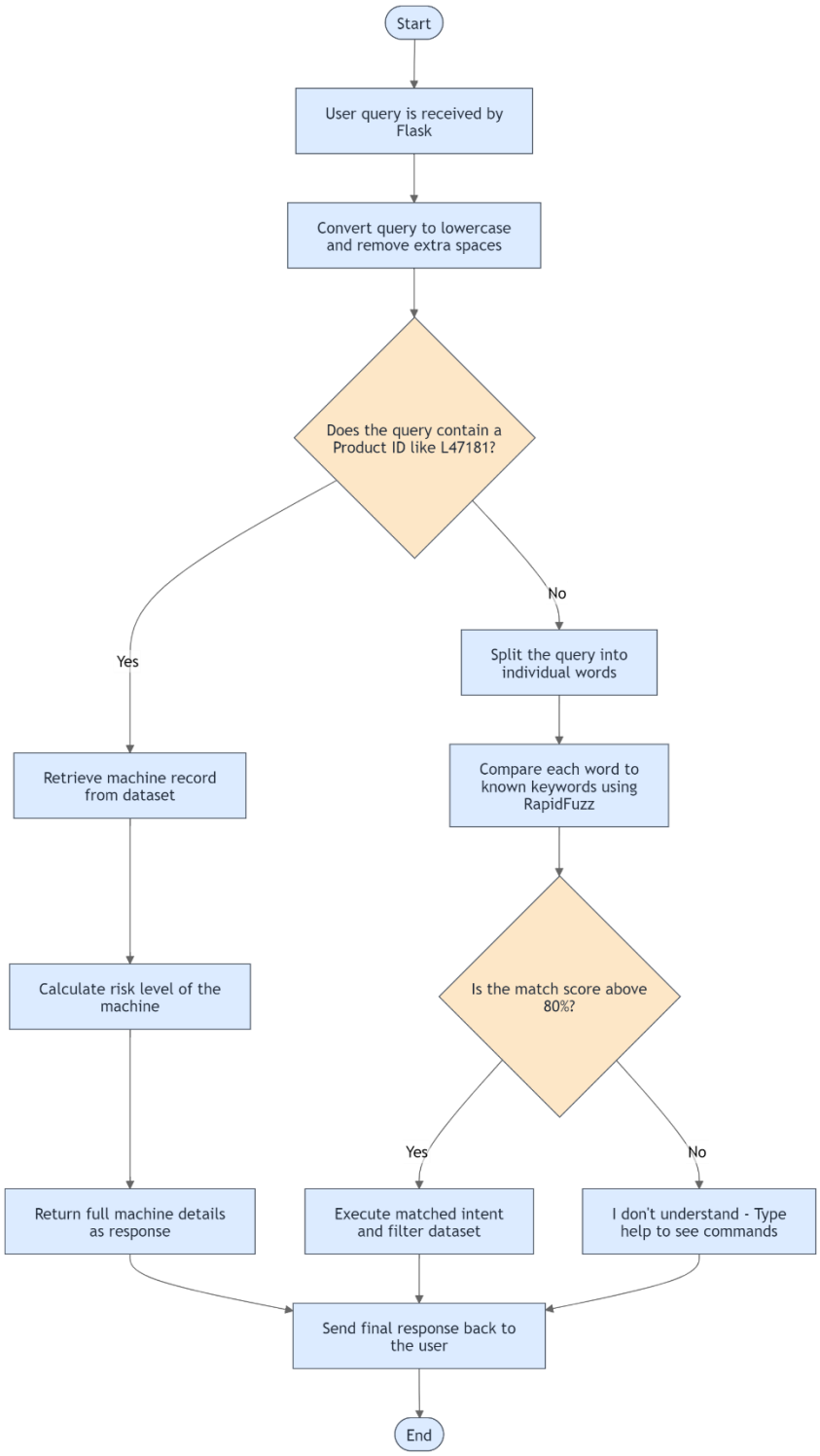
2. System Design

2.1 Application Design

matches() Function:
Performs two-layer matching. First checks exact match using in. If not found, splits query and uses fuzz.ratio() (RapidFuzz). Returns True if similarity ≥ 80%, else False.

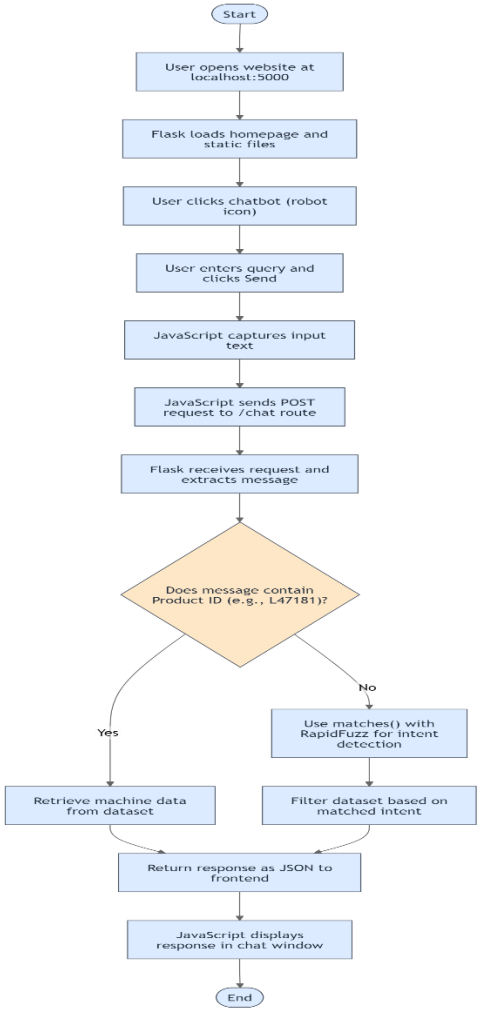
get_response() Function:
Main chatbot function. Normalizes input, detects Product ID using regex, and fetches machine data from DataFrame. If ID is found, returns sensor data with risk status. Otherwise, uses matches() with if-elif logic to identify intent and return response.

STRING MATCHING LOGIC FLOWCART

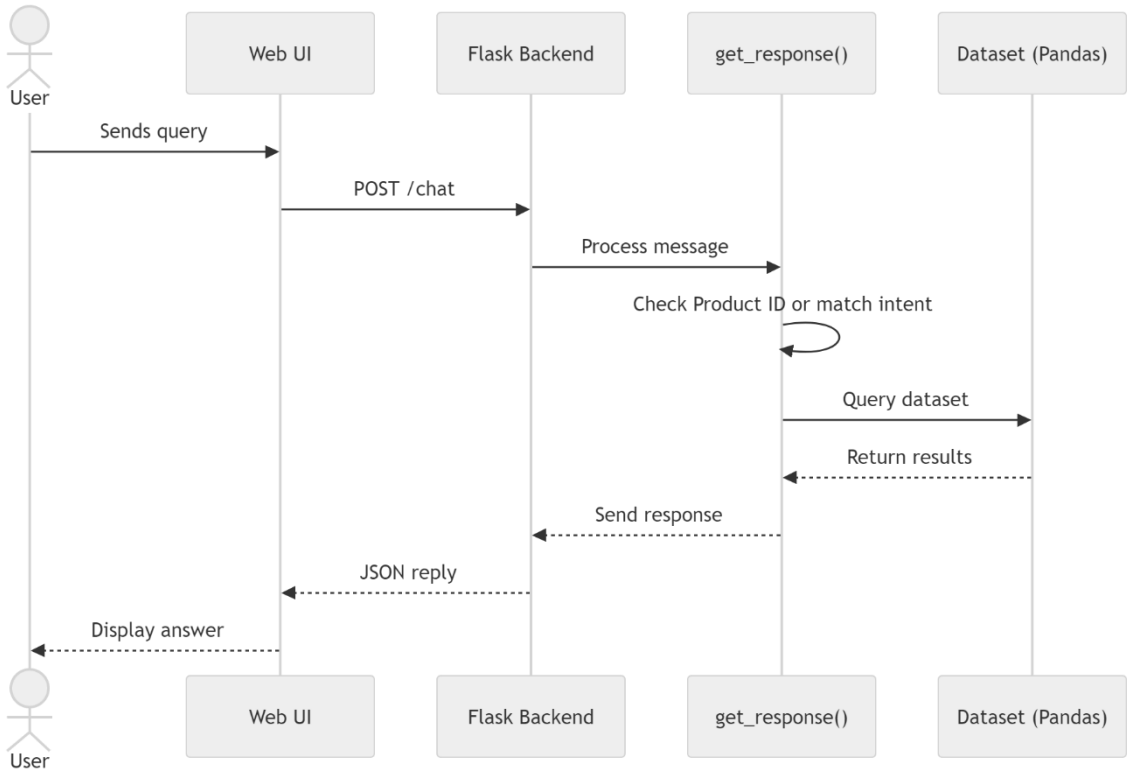


2.2 Process Flow

- User opens the website and accesses the chatbot UI
- Flask serves frontend files (HTML, CSS, JS)
- User clicks chatbot icon and enters a query
- JavaScript captures input and sends POST request to /chat
- Flask receives request and calls get_response()
- System checks for Product ID using regex
- If found, machine data is fetched and risk is calculated
- If not, matches() uses RapidFuzz for intent detection
- Dataset is filtered using Pandas based on condition
- Response is sent back as JSON
- Frontend displays the bot reply in chat window



SEQUENCE DIAGRAM



2.3 Information Flow

The user enters a query in the Web UI, and JavaScript sends it to Flask via a POST request using fetch().

Flask receives the message at /chat and passes it to get_response() for processing. The function applies regex for Product ID detection and fuzzy matching for intent detection, then filters the Pandas DataFrame to get results.

The response is formatted, sent back as JSON, and JavaScript displays it in the chat window after removing the typing indicator.

2.4 Components Design

The system is modularly divided into frontend and dashboard components, each responsible for specific UI handling and data processing tasks.

Frontend Functionality (script.js)

- initSidebar() → Handles sidebar toggle and overlay behavior
- toggleChat() → Opens and closes the chatbot window
- sendMessage() → Captures input, shows typing indicator, sends POST request to /chat, and displays response
- appendMessage() → Renders user and bot messages in the chat window
- scrollChat() → Auto-scrolls to the latest message
- quickQuery() → Triggers predefined queries for quick interaction

Dashboard Functionality (dashboard.js)

- loadDashboard() → Fetches data from /api/dashboard-data
- updateKPIs() → Updates KPI values (total, working, failed, failure rate)
- animateValue() → Animates KPI numbers smoothly
- renderStatusChart() → Displays machine status (doughnut chart)
- renderFailureChart() → Displays failure type distribution (bar chart)
- renderTable() → Renders machine data table with status labels
- renderSpeedGauge() → Displays average RPM gauge
- renderTorqueGauge() → Displays average torque gauge

2.5 Key Design Considerations

- Tiered Matching: Regex is used first for Product ID detection, followed by RapidFuzz for fuzzy keyword matching
- Priority Processing: Machine-specific queries are handled before general intent matching in get_response()
- Data-Driven Thresholds: Risk levels are based on dataset analysis (e.g., 200-minute tool wear for High Warning)
- Dynamic UI Feedback: "Synthesizing Command" indicator improves user experience during processing

2.6 API Catalog

Route	Method	Purpose
/	GET	Loads homepage and chatbot UI
/dashboard	GET	Loads analytics dashboard
/chat	POST	Processes chatbot queries and returns response
/api/dashboard-data	GET	Returns aggregated dataset statistics

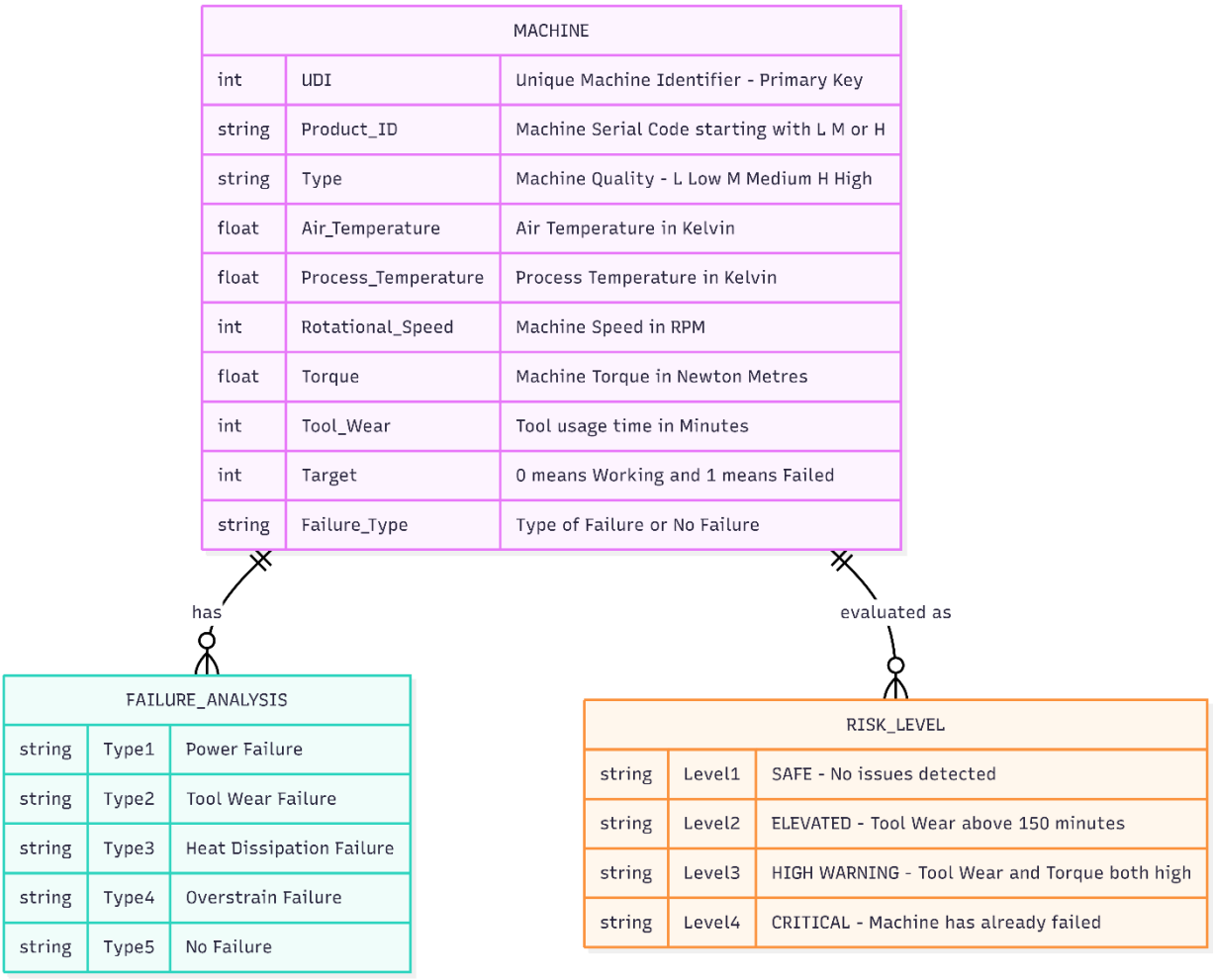
3. Data Design

3.1 Data Model

The data model is based on predictive_maintenance.csv, which serves as the single source of truth for all machine data. The primary entity is **MACHINE**, representing each of the 10,000 records.

The **FAILURE ANALYSIS** entity is derived from the *Failure Type* column and includes six categories: No Failure, Power Failure, Tool Wear Failure, Heat Dissipation Failure, Overstrain Failure, and Random Failures.

The **RISK LEVEL** entity is not stored in the dataset but is dynamically computed using calculate_risk_status(). It classifies machines into four levels: Safe, Elevated, High Warning, and Critical.



3.2 Data Access Mechanism

- Dataset loaded once using pd.read_csv() and stored in memory (DataFrame)
- Queries use Pandas operations (filtering, aggregation)
- Examples: failed machines, failure types, averages, specific machine lookup
- All operations are **read-only**
- Query time: **< 5 ms**

3.3 Data Retention Policies

- No user data stored
- Chat history exists only in browser (clears on refresh)
- Session stores only last Product ID
- Dataset is static and unchanged

3.4 Data Migration

- No migration required (CSV-based system)
- Dataset can be replaced with same structure
- Flask restart required to reload data
- No code changes if schema remains same

4. Interfaces

User Interface

- Homepage: chatbot + informational sections
- Dashboard: KPIs, charts, gauges, data table

Frontend–Backend Interface

- /chat (POST) → query → response
- /api/dashboard-data (GET) → analytics data

Backend–Data Interface

- Uses in-memory Pandas DataFrame
- No database or external APIs

5.State and Session Management

- Stores last Product ID in Flask session
- Supports follow-up queries
- No persistent backend state
- Chat history not stored server-side

6. Caching

- DataFrame acts as in-memory cache
- CSV loaded once at startup
- No external caching tools
- Browser caches static files

7. Non-Functional Requirements

7.1 Security Aspects

- Runs on localhost
- No personal data collected
- Input normalized before processing
- Debug mode only for development

7.2 Performance Aspects

- In-memory processing (no disk I/O)
- Query time: **< 5 ms**
- Response time: **< 500 ms**
- Dashboard load: **< 200 ms (system-dependent)**
- Smooth chart rendering

8. References

- Flask Documentation - https://flask.palletsprojects.com
- Pandas Documentation - https://pandas.pydata.org
- RapidFuzz Library - https://github.com/maxbachmann/RapidFuzz
- Chart.js Documentation - https://www.chartjs.org
- Predictive Maintenance Dataset (Kaggle) - https://www.kaggle.com/datasets/shivamb/machine-predictive-maintenance-classification